项目自读手册 · READER HANDBOOK

对账 Agent 项目手册

把项目从头到尾怎么做的、每一步遇到什么问题、怎么解决、和 AI 怎么互动开发、工作台怎么用，全部用大白话讲一遍。所有数字来自工作日志与成绩单，可复现。

记录期 2026-09-26 至 2026-09-27 · 本地原型 + 合成数据评测

目录

一、这个项目在干什么 2

1.1 三本账的故事 2

1.2 做出来的三样东西 2

1.3 成绩速览 2

二、开发全程：五步怎么走下来 3

2.1 P0：先摸清“天然”有多少对不上 3

2.2 P1：自己出一张带标准答案的考卷 3

2.3 P2：确定性引擎，考卷满分 4

2.4 P3：AI 出场，但戴着镣铐 4

2.5 P4：换个行业的考场再考一遍 4

三、遇到的真实问题，怎么解决 5

3.1 埋在“零运费”上的雷，谁也判不出来 5

3.2 差几分钱，算错还是算对 5

3.3 AI 的“思考循环”：用现成框架还是自己写 5

3.4 界面语言：从“干净/脏世界”到“标准账本/实际账本” 5

3.5 表格里的英文字段：bill_id 是什么 6

3.6 AI 回答把表格挤没了 6

四、这个项目怎么“互动开发” 6

4.1 三份记录，各管一件事 6

4.2 “互动即开发”：三个真实样本 6

4.3 写作纪律：讲明白版 7

五、对账工作台怎么用 7

5.1 证据弹层：三组对照怎么读 7

5.2 AI 问答：能问什么、不能问什么 8

六、这台机器内部怎么搭 9

6.1 两层架构：铁账本 + 会说话的嘴 9

6.2 测试纪律 9

6.3 数据与红线 9

七、诚实的边界 10

附录：十个核心词（人话版） 10

一、这个项目在干什么

先说最核心的一句话：这个项目做的是对账（reconciliation）——把两本或多本账互相核对，找出所有对不上的地方，并且说清每个错在哪里、该找谁。

1.1 三本账的故事

电商生意里同一笔订单会留下三种记录：买家的支付流水（钱真付了多少）、商家的订单账本（底库里这单买了什么、该收多少钱）、平台出的账单（平台说该跟你结算多少钱）。理想情况下三本账应该严丝合缝；现实里它们常常对不上——平台漏开账单、佣金算错、该退的钱没退、同一笔账出现两次……对账员的日常就是把这些问题一笔笔捞出来。

这个项目没有现成的“平台账单”可用（整个行业都拿不到，平台不给开放），所以按照公开的规则卡（佣金率 0.20、运费、退款规则）自己生成平台账单和退款流水；订单和支付则是真实数据——巴西电商公开数据集 Olist 的 9.9 万张订单。这套做法对外一律称“本地原型 + 合成数据评测”，不使用任何公司真实数据。

1.2 做出来的三样东西

- 对账引擎：纯规则的程序，把三本账互相对，自动找出全部差异并分类——不用人工智能，快（9.9 万单约 2.5 秒）、可复现（同数据重跑结果逐位一致）、可审计（每一步留下文件指纹，被改过就断链）。
- AI 归因：大模型只负责两件事——把查实的错误“讲成人话”、回答自然语言提问；它说的每个数字都必须能在账本里找到，对不上就自动作废换模板文案。
- 对账工作台：双击桌面图标打开的本地网页，四个页签翻账本、看差异、问 AI，只读不算账。

1.3 成绩速览

100%	0	99.67%	96 项
1,750 个埋入错误全部检出	真误报（没冤枉一笔好账）	换到银行对账基准的精度	自动化测试全绿（截至 09-27）

跨域成绩的参照物：官方给出的 ChatGPT 方案在同一场考试里精度 95.20%、覆盖率 64.9%，本项目三项指标全面超过，且 1 秒判完 32,048 笔。数字全部来自工作日志与成绩单，可复现。

二、开发全程：五步怎么走下来

项目按五个阶段推进（代号 P0 到 P4），一天半完成主线。每一步都遵守同一个节奏：先想清为什么做、做完必须验证，验证不过不放行。总览如下，后面逐步展开。

阶段	做了什么	关键结果
P0 摸家底	真实数据装库，盘点“天然”对不上的账	99.42% 一致；576 单天然差异
P1 出考卷	生成平台账单，埋 1,750 个已知错误	冻结考卷 v1.1 + 标准答案
P2 写引擎	规则引擎对三本账、分类差异	1,750 颗雷全中、零真误报
P3 上 AI	受约束的归因解释与问答	答案数字全过白名单校验
P4 换考场	同一套思想考银行对账基准	精度 99.67%，超官方基线

表 1 五个阶段的里程碑

2.1 P0：先摸清“天然”有多少对不上

做了什么：把 9 张原始表装进本地数据库，全面盘点“订单账本 vs 支付流水”天然对不上的地方。结果：99.42% 的订单天然一致，576 单金额对不上——大多是几分钱的尾差，属于正常现象。

为什么必须先做：不先划清“天然错误”，后面人工埋的雷就无法判分——引擎报出一条差异，你分不清它是抓到了埋的雷，还是抓到了天然尾差。这一步等于给考场先划定“地面本来就有几道划痕”，埋雷与判分才有干净的参照。

怎么验证的：9 张表行数与原始文件逐张一致；关键数字用两套独立写的查询互核，两边吻合。

顺带发现：数据库建表时故意不给订单号加“唯一”约束——约束会静默丢掉重复行，而对账项目里“重复”本身就是要测量的错误。数据还没开始对，地基不能先歪（决定记录 D001）。

2.2 P1：自己出一张带标准答案的考卷

做了什么：写“造数五件套”——平台账单生成器、退款流水生成器、六类埋雷器、标准答案读写、编排器（带自校验）。对 9.9 万单真实底座跑出冻结评测集：干净三本账 + 被埋了雷的三本账 + 1,750 条标准答案（每条写清雷在哪、什么类型）。

为什么要“冻结（freeze）”：考题和标准答案定稿封存之后，引擎怎么改都考同一套题，分数才可比——今天 100 分、明天 98 分，就说明改动真把某类题改坏了。

怎么验证的：22 个自动化测试；造数脚本内置自校验（行数守恒、天然差异单绝不被埋雷），不过就报错退出；关键数字与 P0 盘点对得上。

2.3 P2：确定性引擎，考卷满分

做了什么：引擎四件套——三本账装载器、六类规则树分类器、独立复核器、审计链。判卷成绩一句话概括：1,750 颗雷全部检出、真误报 0（即召回 100%）；另报出的 575 条天然差异单独分桶、不计错报，全量判完约 2.5 秒。

为什么坚持不用 AI 做匹配：匹配要可复现、可审计、可判分，AI 三样都不占；把 AI 放在主匹配路径上，结论就没法让人放心。AI 的价值在“组织语言”，不在“找账”。

怎么验证的：51 个测试；复核器不读引擎结论、只凭订单号和差异类型从原始数据独立重推，2,325 条发现全部复核通过；审计链校验通过。首跑曾经翻车（见第三章第一节），修好后考卷重冻结、满分。

2.4 P3：AI 出场，但戴着镣铐

做了什么：AI 只出现在两个受控出口——每条差异的“归因解释”按钮、概览页的问答框。答案头顶有标签：“AI · 已过白名单校验”。真模型冒烟测试通过：归因一条删支付的雷，引用编号齐全；问它事实单里没有的信息，它如实回答“事实单未提供”。

核心设计：喂给模型的只有“事实单”（这条差异相关的三张表摘录），提示词限定“只能用事实单、不许算新数字“；答案回来后逐个核对数字与编号是否真的出现在事实单里——违规自动重写一次，再违规直接换成写好的确定性文案。详见第六章。

怎么验证的：96 个测试中的 31 个属于 Agent 与工作台接口，全部用假模型离线跑，永不碰网络；真模型只做冒烟演示，不进测试。

2.5 P4：换个行业的考场再考一遍

做了什么：把同一套匹配思想搬到 BenchRec——一个真实的银行流水对账公开基准（有官方判分和官方 ChatGPT 基线）。结果：精度 99.67%、覆盖 84.6%、判对率 90.4%，1 秒判完 32,048 笔，全面超过官方基线（95.20% / 64.9% / 65.2%）；距竞赛 99.8% 的硬门槛差 0.13 个百分点。

为什么值得做：主战场成绩再好也可能被问“是不是只在自己造的数据上好用”。换一个行业的数据集重考，是“方法通用”的直接证据——这正是“跨域验证（cross-domain）”的含义。

诚实记录：89 个错配逐类归因成文；一个改进方向被实验证伪（参考号字面印证：精度能过线，覆盖率却崩到 18.6%，因为两个数据集的编号是不同体系）——失败的实验也写进报告，防止后人重走弯路。

三、遇到的真实问题，怎么解决

以下六个问题全部来自项目的工作日志与决定记录，原汁原味。每个按“问题 → 怎么选 → 结果”讲。

3.1 埋在“零运费”上的雷，谁也判不出来

引擎首次全量跑分，1,750 颗雷竟漏了 2 颗。追查发现：这 2 个订单的明细运费本来就是 0，“把 0 改成 0”的雷物理上不可判定——任何引擎都抓不出来。

怎么选：要么带着缺陷上（报告注明“检出上限 99.89%”），要么修造数器、整卷重出、重新冻结。选了后者：给埋雷器加硬校验，漏运费雷只能埋在运费大于 0 的订单上；同种子重出考卷（v1.1），3 分钟完成，数量口径不变，引擎重跑满分。

教训：小样本测试（运费全为正）盖不住全量的边角情况；“每颗雷必须可判定”这种语义层面的自校验要写进脚本，不能靠人眼。（决定记录 D011）

3.2 差几分钱，算错还是算对

P0 盘点发现 576 单金额不一致，其中 477 单差值在 10 雷亚尔以内——多是四舍五入的尾差。容差定多严？定死了，全是假警报；定松了，真错误溜走。

怎么选：金额比较容差 ±0.005 雷亚尔——即比较到“分”（当地货币最小单位）。能抓住真实错误、放过舍入噪音。若跨域考试发现大量 1 分位误报，再升到 ±0.01 并记录影响。（D002）

3.3 AI 的“思考循环”：用现成框架还是自己写

AI 归因需要“思考 → 查工具 → 再思考”的循环。现成框架（LangGraph）功能全，但本项目 AI 只有两个受控出口、单任务最多 8 步、5 到 6 个工具，框架的重能力（断点恢复、人工介入）全用不上。

怎么选：自己写约 300 行的受约束循环，每一行护栏都看得见：工具白名单、步数预算、操作留痕、答案查引用、失败就说不知道。上一项目用过框架，这一项目自研——“会用框架、更懂框架底下是什么“。（D006）

3.4 界面语言：从“干净/脏世界”到“标准账本/实际账本”

工作台初版页签叫“干净世界 / 脏世界”——内部开发词直译上界面，用户第一眼就看不懂。界面全部改成自然中文：标准账本（账本应该长这样）/ 实际账本（平台实际给的账）、差异明细、埋下错误。只改显示文字，代码字段与接口一律不动。

同一轮还发现浏览器缓存旧页面导致“改了没生效”，根治办法是给页面资源加版本号参数（?v=N），每改一次前端就升一位。

3.5 表格里的英文字段：bill_id 是什么

用户点开证据弹层：“bill_id、payment_sequential 这样直接看都看不懂。”于是字段名全部中文化（bill_id → 账单号、payment_sequential → 支付笔次、order_status → 订单状态），枚举值也翻译（credit_card → 信用卡、delivered → 已送达）。

边界：订单号、账单号这类编号和金额故意不翻——编号是账本里的“身份证号”，两边对账靠它一字不差；翻成中文反而没法核对。这个“翻译有边界”的决定，比全翻更有对账味。

3.6 AI 回答把表格挤没了

用户点“AI 归因解释”后反馈：回答一旦展开就收不回去，把上面看订单表格的空间挤掉了。修改：按钮改为开关式，点一下展开、再点一下收起；同一订单的回答按单缓存，收起再展开不重复调用模型；每次新开弹层自动回到收起状态。

为什么这么改：弹层的主角是三组账本对照，AI 解释只是辅助——辅助信息不能抢占主信息的空间。这个问题开发时完全没想到，是真实试用暴露出来的。（这也说明：试用本身就是开发。）

四、这个项目怎么“互动开发”

项目采用“放手让 AI 干活”的模式：AI 负责执行，人负责提问、试用、拍板方向。人不可能盯每一步代码，所以记录就是眼睛——想知道“为什么做这步”，答案永远在记录里，不用翻聊天记录。

4.1 三份记录，各管一件事

记录	管什么	举例
工作日志	每件事四栏：做了什么 / 为什么 / 怎么验证 / 遗留	本章开头引用的每一条
决定记录	重要取舍：问题 → 备选 → 选了哪个 → 何时推翻	12 条（D000 至 D012）
术语卡	黑话与人话对照，一行打住	附录收录了 10 个核心词

表 2 记录体系的分工

4.2 “互动即开发”：三个真实样本

- 提问即测试：用户连着问“单边账是什么”“原始记录是什么”——每个问题都在检验界面能不能把事情讲明白；答案直接决定了术语翻译和对照排版怎么改。
- 试用即需求：用户说“想看输入的账本本身”，催生了账本浏览页（六本账通读、搜索、翻页）和弹层三组对照；说“看不懂字段”，催生了中文化；发现“回答收不回去”，催生了开关式按钮。
- 反馈即验证：每轮改动都回到浏览器里实测（展开、收起、翻页、搜索），96 项自动化测试做后盾，改动当天提交入库，可回溯。

4.3 写作纪律：讲明白版

所有记录遵守同一套写作法（D007）：黑话只留个名，格式是“人话（黑话）”，第一次出现就解释；每篇解释固定四段（卡在哪 → 做了什么 → 不这么做会怎样 → 面试怎么说）；事实、决定、猜测、卡点分开写；估算标明“这是估算”。本手册也是按这套规矩写的。

五、对账工作台怎么用

双击桌面（或项目目录）的“对账工作台”图标，几秒后浏览器自动打开本地页面（只在本机运行，外面访问不到）。四个页签，从总览到细节：

- 概览：评测集总量、六类差异分布、三本账行数、AI 问答框。顶部两张“账本卡”点击直达账本浏览。
- 差异明细：每一条差异一行，可按类型筛选；点任意一行打开证据弹层。
- 账本浏览：六本账（标准 / 实际 × 平台账单 / 支付 / 退款）通读，支持搜索和翻页，点行跳证据弹层。
- 关于：架构图、里程碑、口径说明。

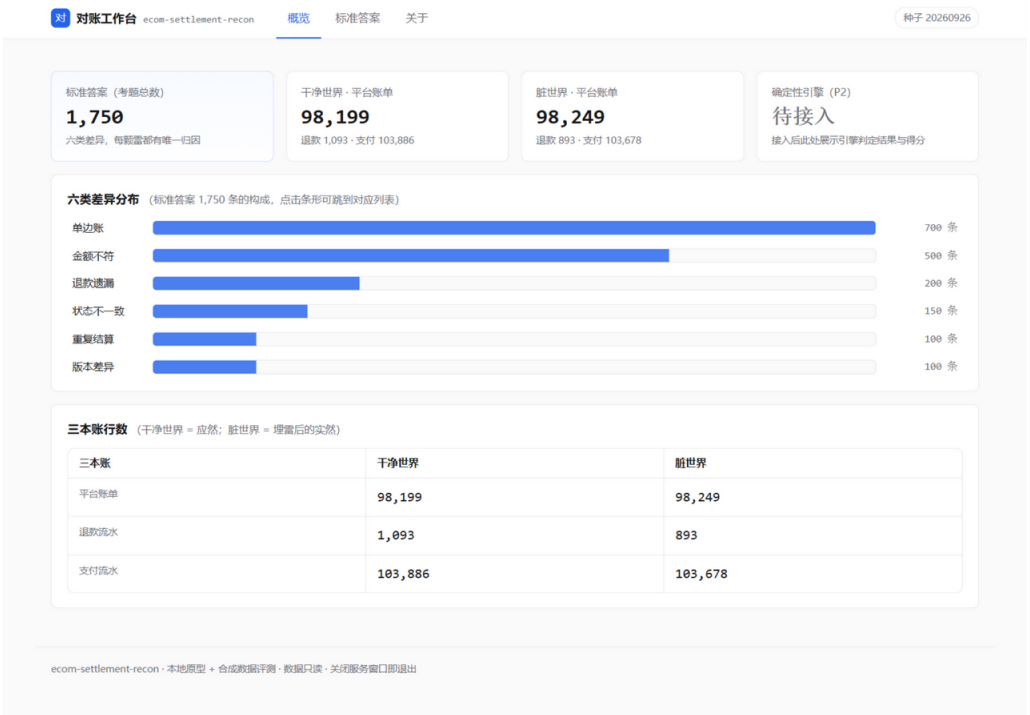


图 1 概览页：左侧四张数字卡，中部六类差异分布（点击条形跳列表），下方三本账行数

5.1 证据弹层：三组对照怎么读

点开任何一条差异，弹层从上到下三组：标准账本（这个订单应有的账）、实际账本（平台实际给的账）、原始记录（订单底单，谁都没动过的档案）。以一条“单边账”为例读法：标准账本里平台账单 1 行、支付流水 1 行；实际账本里平台账单 0 行（红条标注“这正是埋下错误的位置”）——缺的是平台账单这一边。

再看原始记录做交叉验证：明细单价 159 + 运费 27.19 = 186.19，与支付流水的支付金额分毫不差——生意是真的、钱也是真的，唯独账单没了，所以该找平台补账单。缺哪边，决定该找谁。

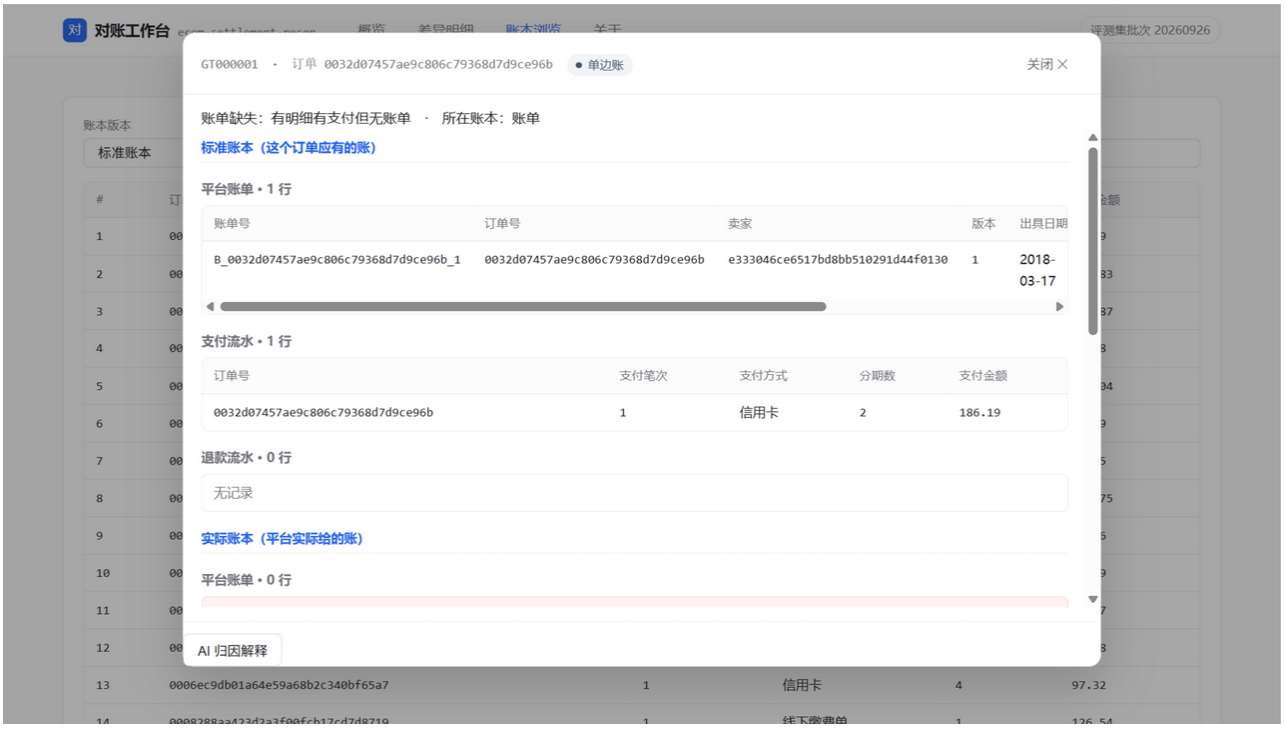


图 2 证据弹层：字段名与状态值已全部中文化，红条标出埋错位置

5.2 AI 问答：能问什么、不能问什么

概览页问答框和弹层里的“AI 归因解释”走同一套约束。能问：六类差异的例子（“单边账有哪些？”）、引擎成绩（“考得怎么样？”）、粘贴一个 32 位订单号查它的账。不能指望：开放聊天、让它算新数字、问账本里没有的信息——它要么用固定模板回答，要么如实说“事实单未提供”，不会编。



图 3 AI 问答：回答带来源标签，数字必须能对回账本

六、这台机器内部怎么搭

6.1 两层架构：铁账本 + 会说话的嘴

底层：确定性引擎。纯规则程序，按“同币种 + 账户 + 金额精确分桶 → 日期贴近排序 → 参考号消歧 → 无法消歧就放弃“的思路匹配三本账。快、可复现、可审计。每一步的输入输出都留文件指纹（哈希），环环相扣成审计链——任何一环被改，链条立刻断裂，结论自动作废。

上层：受约束 AI。大模型只在两个出口出现，只许组织语言、不许算账。防止它“一本正经地编数字”靠三道闸：提示词限定“只能用事实单”；输出白名单校验（每个数字和编号必须能在证据里找到，违规重写一次）；再违规降级为写好的模板文案。

6.2 测试纪律

96 项自动化测试全部离线（给 AI 换成假模型，永不碰网络、不花钱、毫秒级跑完）；真模型只做冒烟演示。每个功能先写测试、看它失败、再实现到通过——测试红过，才证明它真的在测东西。

6.3 数据与红线

- 数据来源：Olist 公开电商数据集（9.9 万单）+ 按公开规则卡合成的平台账单与退款流水；跨域用 ICAIF 2023 银行对账基准 BenchRec。不含任何公司真实数据。
- API 密钥只放本机环境变量，不进代码、不进仓库、不进日志。
- 对外表述统一为“本地原型 + 合成数据评测”。

七、诚实的边界

一个可信的系统要能说清自己不能做什么。当前已知边界：

- AI 问答对“统计类词”会直接走模板，不进入开放对话；没有多轮对话，没有让 AI 自主决定调用工具的循环。
- 跨域成绩距竞赛 99.8% 硬门槛差 0.13 个百分点；89 个错配已逐类归因（最大一类是“金额唯一候选无印证即配“，另含数据集官方标注本身 0.2% 的噪声），改进方向已列（占用约束 / 长编码印证 / 训练阈值）。
- 平台账单按公开规则卡合成，佣金率（0.20）不追求行业真实——引擎抓的是“与规则不符”，不是“与行业不符”。

待定事项一条：是否继续追那 0.13 个百分点。当前判断是边际收益低于收尾交付，先不做；要改先在决定记录立项。

> 如果只记三句话，请记这三句：先摸清天然错误再埋雷；确定性引擎管找账、AI 只管说话；每个数字都要能对回账本。

附录：十个核心词（人话版）

词	一句人话
对账（reconciliation）	把两本或多本账互相核对，找出对不上的地方。
三方对账	核对三本账：买家支付流水、商家订单账本、平台账单。
埋雷（injection）	故意往干净数据里制造已知错误，当考试题。
标准答案（ground truth）	埋雷时记下的文件：每颗雷在哪、什么类型，用来判分。
评测集冻结（freeze）	考题和答案定稿封存，之后怎么改都考同一套，分数才可比。
容差（tolerance）	允许的误差范围；本项目金额比较到“分”（±0.005 雷亚尔）。
审计链（audit trail）	一环扣一环的操作记录，每环带文件指纹，被改就断链。
护栏（guardrails）	给 AI 划的行动边界：工具白名单、步数预算、留痕、查引用、失败就说不知道。
跨域验证（cross-domain）	同一套方法换个行业的账再考一遍，证明通用。
薄启动器（launcher）	只负责拉起本机服务和开浏览器的小 exe（7MB），代码数据留在项目目录。

表 3 术语速查（完整版见项目 glossary.md）